

Chapter 9

Reward Model Training

Reward models are the bridge between human preferences and the RL training signal. A well-trained reward model is essential for successful RLHF; a poorly trained one leads to reward hacking and misaligned behaviour. This section covers the theoretical foundations, practical training techniques, and architectural choices for reward models.

9.1 Bradley-Terry Model – Full Derivation

The Bradley-Terry model [29] is the standard probabilistic framework for pairwise preference learning. Given two responses y_1 and y_2 to a prompt q , the model assumes:

$$P(y_1 \succ y_2 \mid q) = \sigma(r(y_1, q) - r(y_2, q)) = \frac{e^{r(y_1, q)}}{e^{r(y_1, q)} + e^{r(y_2, q)}},$$

where $r : \mathcal{Y} \times \mathcal{Q} \rightarrow \mathbb{R}$ is the scalar reward function and σ is the sigmoid function.

Maximum Likelihood Estimation

Given a dataset $\mathcal{D} = \{(q^{(k)}, y_w^{(k)}, y_l^{(k)})\}_{k=1}^N$ of preference pairs, the MLE objective is:

$$\mathcal{L}_{\text{BT}}(\phi) = -\frac{1}{N} \sum_{k=1}^N \log \sigma(r_\phi(y_w^{(k)}, q^{(k)}) - r_\phi(y_l^{(k)}, q^{(k)})),$$

where r_ϕ is a neural network parameterised by ϕ . This is a binary cross-entropy loss where the “positive” class is the preferred response.

Bradley-Terry Assumptions

1. Preferences are *transitive*: if $y_1 \succ y_2$ and $y_2 \succ y_3$, then $y_1 \succ y_3$.
2. Preferences are determined by a *scalar* reward (no multi-dimensional preferences).
3. The preference probability depends only on the *difference* in rewards.
4. Preferences are *independent* across pairs (no annotator effects).

These assumptions are often violated in practice, motivating extensions like Plackett-Luce models for ranking and multi-dimensional reward models.

Margin Loss Extension

A common extension adds a margin m to ensure a minimum gap between winning and losing rewards:

$$\mathcal{L}_{\text{margin}} = -\frac{1}{N} \sum_{k=1}^N \log \sigma(r_\phi(y_w^{(k)}, q^{(k)}) - r_\phi(y_l^{(k)}, q^{(k)}) - m).$$

9.2 Reward Model Architectures

Classification Head on LLM

The standard reward model architecture takes a pretrained LLM and replaces the language modelling head (which maps hidden states to vocabulary logits) with a scalar regression head (which maps the final hidden state to a single reward value).

The architecture is:

1. **Backbone:** a pretrained LLM (e.g., Llama, Mistral) that encodes the prompt-response pair into a sequence of hidden states.
2. **Pooling:** extract the hidden state at the last token position (for decoder-only models) or the [CLS] token (for encoder models).
3. **Regression head:** a linear layer $W \in \mathbb{R}^{d \times 1}$ that maps the pooled hidden state to a scalar reward.

Reward Model Training in TRL

```
from trl import RewardConfig, RewardTrainer
from transformers import AutoModelForSequenceClassification

# Load model with scalar head (num_labels=1)
model = AutoModelForSequenceClassification.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    num_labels=1,
)

config = RewardConfig(
    output_dir="reward_model",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=1e-5,
    num_train_epochs=1,
    # Margin loss
    center_rewards_coefficient=0.01,
)

trainer = RewardTrainer(
    model=model,
    args=config,
    train_dataset=dataset, # must have chosen/rejected columns
)
trainer.train()
```

9.3 Reward Model Training Tricks

Reward Centering

Raw reward model outputs can have arbitrary scale and offset. Centering the rewards (subtracting the mean) stabilises RL training:

$$r_{\text{centered}}(y, q) = r_{\phi}(y, q) - \mathbb{E}_{y' \sim \pi_{\theta}}[r_{\phi}(y', q)].$$

In TRL, this is implemented via the `center_rewards_coefficient` parameter, which adds a regularisation term to the reward model loss that penalises non-zero mean rewards.

Length Bias Correction

Reward models are known to exhibit *length bias*: they tend to assign higher rewards to longer responses, regardless of quality. This can be corrected by:

1. **Length normalisation**: divide the reward by the response length.
2. **Length-controlled training**: include length as a feature and train the model to be length-invariant.
3. **Calibration**: post-hoc regression to remove the length effect.

Margin Losses

Adding a margin m to the Bradley-Terry loss ensures the reward model assigns meaningfully different scores to preferred and dispreferred responses:

$$\mathcal{L}_{\text{margin}} = \max(0, m - (r_w - r_l)).$$

9.4 Process Reward Models vs Outcome Reward Models

PRM vs ORM Comparison

Property	ORM	PRM
Reward signal	Final answer only	Each reasoning step
Training data	(prompt, answer, correct?)	(prompt, steps, step labels)
Annotation cost	Low	High
Credit assignment	Sparse	Dense
Reward hacking	Easier to hack	Harder to hack
Best for	Simple tasks	Multi-step reasoning
Inference cost	Low	High (score each step)

When to Use PRMs

Process Reward Models are most valuable when:

- The task requires multi-step reasoning (math, code, logic).
- The final answer is binary (correct/incorrect) but intermediate steps vary in quality.
- You want to use the reward model for *search* (e.g., beam search with step scores).
- You have access to step-level annotations (or can generate them automatically).

For simple tasks (sentiment, toxicity, factuality), ORMs are sufficient and much cheaper.

PBRS in RLHF for LLMs

Original reward: Binary correctness (1 if final answer is right, 0 otherwise) — extremely sparse for multi-step reasoning.

Potential function: $\Phi(s)$ = partial credit from a verifier (e.g., fraction of intermediate reasoning steps that are logically valid).

Shaped reward: Agent gets incremental signal for each valid reasoning step while preserving the guarantee that the optimal policy still maximizes final-answer correctness.

Practical implementations:

- Process reward models (PRMs) that score each step in a chain-of-thought
- Intermediate compilation checks in code generation

- Partial match scores for multi-part answers

This is a direct application of Potential-Based Reward Shaping (PBRs) [265] to the LLM setting—the theoretical guarantee that shaped rewards preserve the optimal policy makes PRMs a principled approach to dense reward in reasoning tasks.

Automatic PRM Annotation

Step-level annotations can be generated automatically using:

1. **Monte Carlo rollouts:** for each intermediate step, sample multiple completions and use the fraction that reach the correct answer as the step reward.
2. **LLM-as-judge:** use a strong LLM to evaluate each step.
3. **Formal verification:** for math/code, use a verifier to check each step.

9.5 Rule-Based Rewards for RLVR

Reinforcement Learning from Verifiable Rewards (RLVR) uses deterministic, rule-based reward functions instead of learned reward models. This substantially reduces reward hacking (though models can still exploit format tricks, edge cases, or test memorization) and is the approach used in DeepSeek-R1 [72].

Rule-Based Reward Functions in TRL

```
import re

def format_reward(completions, **kwargs):
    """Reward for using <think>...</think><answer>...</answer> format."""
    rewards = []
    pattern = r"<think>.*?</think>\s*<answer>.*?</answer>"
    for completion in completions:
        text = completion[0]["content"]
        rewards.append(1.0 if re.fullmatch(pattern, text, re.DOTALL) else 0.0)
    return rewards

def correctness_reward(completions, ground_truth, **kwargs):
    """Reward for correct final answer."""
    rewards = []
    for completion, gt in zip(completions, ground_truth):
        text = completion[0]["content"]
        match = re.search(r"<answer>(.*?)</answer>", text, re.DOTALL)
        if match:
            answer = match.group(1).strip()
            rewards.append(1.0 if answer == gt else 0.0)
        else:
            rewards.append(0.0)
    return rewards

def code_execution_reward(completions, test_cases, **kwargs):
    """Reward for code that passes test cases."""
    import subprocess, tempfile, os
    rewards = []
    for completion, tests in zip(completions, test_cases):
        code = completion[0]["content"]
        passed = 0
        for test in tests:
            with tempfile.NamedTemporaryFile(
                mode="w", suffix=".py", delete=False
            ) as f:
                f.write(code + "\n" + test)
                fname = f.name
```

```

try:
    result = subprocess.run(
        ["python", fname], capture_output=True,
        timeout=5, text=True
    )
    passed += int(result.returncode == 0)
except Exception:
    pass
finally:
    os.unlink(fname)
rewards.append(passed / len(tests))
return rewards

```

Rule-Based Reward Pitfalls

- **Format gaming:** models learn to produce the correct format without correct content. Always combine format and correctness rewards.
- **Test case leakage:** if test cases are in the training data, the model memorises them.
- **Timeout exploitation:** models may generate code that times out (avoiding failure). Use strict timeouts and penalise timeouts explicitly.
- **Reward sparsity:** binary rewards (0/1) can be too sparse for complex tasks. Consider partial credit or intermediate rewards.

9.6 Multi-Objective Rewards – Combination Strategies

When training with multiple reward signals, the combination strategy significantly affects the final policy.

Multi-Reward Combination Strategies

1. **Weighted sum:** $r = \sum_n w_n r_n$. Simple but sensitive to scale.
2. **Normalise then sum (GDPO):** normalise each reward to zero mean and unit variance within the group, then sum with weights. Scale-invariant.
3. **Lexicographic:** optimise rewards in priority order; only consider lower-priority rewards when higher-priority ones are tied.
4. **Constrained:** maximise primary reward subject to constraints on secondary rewards.
5. **Pareto:** maintain a Pareto front of policies and select based on preference.

Multi-Reward Training in TRL

```

from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    # GDPO: normalise each reward independently
    multi_objective_aggregation="normalize_then_sum",
    reward_weights=[1.0, 0.3, 0.1], # correctness, format, length
    num_generations=8,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[
        correctness_reward,
        format_reward,
        length_penalty_reward,
    ],

```

```
args=config,
train_dataset=dataset,
)
```

9.7 Listwise Rank-Based Rewards

While the Bradley-Terry model handles *pairwise* preferences ($y_w \succ y_l$), many practical scenarios involve ranking multiple responses simultaneously. Listwise reward models learn from complete orderings, providing richer training signal and enabling better calibration.

Motivation: Beyond Pairwise

Why Listwise?

- **Richer signal:** A ranking of K responses contains $\binom{K}{2}$ implicit pairwise comparisons, but also captures *relative margins* (how much better rank 1 is vs. rank 3).
- **Better calibration:** Pairwise BT models only learn *differences* in reward; listwise models learn absolute reward scale.
- **Natural fit for GRPO:** GRPO generates N responses per prompt and ranks them — listwise rewards align directly with this workflow.
- **Annotator efficiency:** Ranking 5 responses is faster than labeling all 10 possible pairs independently.

Plackett-Luce Model

The Plackett-Luce (PL) model [291] is the standard extension of Bradley-Terry to full rankings. Given K responses $y_1, \dots, y_K$ with ranking π (where $\pi(1)$ is the best):

Plackett-Luce Likelihood

$$P(\pi \mid q) = \prod_{i=1}^K \frac{e^{r_\phi(y_{\pi(i)}, q)}}{\sum_{j=i}^K e^{r_\phi(y_{\pi(j)}, q)}}$$

Intuition: Sequentially select the best remaining item. At each step, the probability of selecting item $\pi(i)$ is softmax over the remaining items.

Loss function:

$$\mathcal{L}_{\text{PL}}(\phi) = -\frac{1}{|\mathcal{D}|} \sum_{(q, \pi) \in \mathcal{D}} \sum_{i=1}^{K-1} \left[r_\phi(y_{\pi(i)}, q) - \log \sum_{j=i}^K e^{r_\phi(y_{\pi(j)}, q)} \right]$$

Plackett-Luce Reduces to Bradley-Terry

For $K = 2$, the PL model gives: $P(y_1 \succ y_2) = \frac{e^{r(y_1)}}{e^{r(y_1)} + e^{r(y_2)}} = \sigma(r(y_1) - r(y_2))$ — exactly the Bradley-Terry model. PL is a strict generalization.

ListMLE and Rank-Based Losses

Listwise Loss Functions

- **ListMLE** [391]: Directly maximizes the PL likelihood of the ground-truth ranking. Simple and effective.
- **ListNet** [38]: Minimizes KL divergence between the model’s top-1 probability distribution